

FIREBASE FIRESTORE MASTER TUTORIAL (COMPRESSED EDITION)

This is a structured Firestore tutorial tailored to your booking system project.

It is a condensed version designed to fit output limits, but it contains the full structure for a multi-day, 150k-word curriculum. You can request expansions of any chapter to generate additional PDF volumes.

DAY 1 — FIRESTORE FUNDAMENTALS

- What Firestore is
- How NoSQL works
- Collections, documents, fields, subcollections
- When to use Firestore vs Realtime Database
- Firestore limits and pricing model
- Understanding Firestore regions and latency

ESSENTIAL SDKs

- Firebase Web SDK (client)
- Firebase Admin SDK (server)
- Authentication flow
- Service accounts and private keys

DAY 1.2 — YOUR PROJECT'S DATA MODELING

CUSTOMERS:

- Doc ID = phone number
- Tracks firstSeenAt, lastSeenAt, reservationCount

RESERVATIONS:

- Auto ID per booking
- Stores customerId, email, guests, dateTimeUTC, notes, slotId

SLOTS:

- Document ID = date_time
- Tracks totalGuests, maxGuests, updatedAt

Why this model works:

- Fast lookups
- Predictable write patterns
- Easy aggregation
- Minimal cost

DAY 1.3 — FIRESTORE OPERATIONS

Basic operations:

- setDoc
- updateDoc
- addDoc
- deleteDoc
- serverTimestamp()
- increment()

Transactions:

- How Firestore handles concurrency
- Why your slot system MUST use transactions
- Example race condition and fix

Security rules overview:

- read, write, allow, deny
- request.auth, resource.data
- Schema validation inside rules

DAY 2 — BUILDING YOUR BOOKING API

What your API must do:

1. Validate email (must include "@" & end with ".com")
2. Validate date (must be $\geq$ today: 2025-12-03)
3. Validate time (must be real HH:mm and not in past for today's date)
4. Run a Firestore transaction to update slot capacity
5. Create / update customer
6. Create reservation entry
7. Send confirmation email using Resend
8. Return JSON to frontend

Error handling:

- Slot full
- Invalid email
- Invalid date
- Invalid time
- Firestore commit failures

DAY 2.2 — FIREBASE ADMIN SDK DEEP DIVE

- Service account keys
- Initializing admin inside Next.js API routes
- How admin privileges bypass security rules
- Using `FieldValue.increment`
- Using `serverTimestamp`
- Structure of Timestamp objects

DAY 2.3 — RESEND EMAIL WORKFLOW

- What Resend is
- How to integrate in Next.js App Router
- Common errors (invalid sender domain, inactive API key)
- Best practices for booking confirmation emails

DAY 2.4 — FRONTEND CONNECTION

Your booking UI → POST /api/reservations:

- Input validation
- Formatting date → YYYY-MM-DD
- Handling errors gracefully
- Displaying confirmation UI
- Showing slot availability

DAY 3 — ADVANCED FIRESTORE ARCHITECTURE

Indexes:

- Single-field indexes
- Composite indexes
- Querying reservations by date/time
- Index cost considerations

Admin dashboard features:

- Real-time reservation table
- Slot capacity view
- Customer lookup by phone
- Reservation cancellation logic

Performance:

- Cold-start behavior
- Firestore network flow

- Using cache and persistence

Appendix — Tailored Notes Based on Your Uploaded Files

- Next.js App Router structure fits Firestore well
- Recommended `/app/api/reservations/route.ts` organization
- Your database hierarchy screenshot matches best practices
- Slot IDs using `date_time` format match Firestore indexing strengths

This PDF is the base module. You can request:

- The full 150k word expansion
- Volume 2 (Security Rules Deep Dive)
- Volume 3 (Admin Dashboard Build)
- Volume 4 (Resend + Automations)
- Volume 5 (Advanced Firestore Engineering)